



Errata 6/11/08

page 172. The final line of the code reads "return TestAABBPlane(b, p);" but this is not correct unless we translate the AABB b to the origin (which the given code does not do). To correct this, the AABB should be centered at the origin first, with something equivalent to:

```
AABB b2(b.min - c, b.max - c);
return TestAABBPlane(b2, p);
```

This error was pointed out by Gayathri Chittiappa.

page 174. The third step of the "ERIT" triangle-triangle intersection test (at the bottom half of the page) refers to "triangle 2" but should refer to "triangle 1". The sentence immediate following step 3 should also be referring to triangle 1 instead of triangle 2.

Tiago Nobrega reported this typo.

page 328. The if-statement that reads "if (ty <= tx && ty <= tz)" has a superfluous condition. It should simply read "if (ty <= tz)".

This error was reported by Joey Hammer (PixelActive).

Errata 9/11/07

page 210. At the bottom of the page is a sentence that starts: "It should be noted...". This sentence should be removed as it is incorrect.

Thanks to Joey Hammer (PixelActive) for reporting this.

Errata 3/31/07

pages 226-227. The first method described in Section 5.5.6 (starting at the section start and ending at the second to last paragraph on page 227) is, unfortunately, fundamentally broken. The key problem is the sentence that straddles the the two pages, which reads: "If the moving sphere hits the triangle at all, then Q is the point it will hit first (as the circular cross section of the sphere, as it moves through the plane, will be centered at P)." This is just not true, and what follows therefore is not true either. Instead of using this incorrect method, the second method (by [Schroeder01]) mentioned in the concluding paragraph of page 227 should be used.

This problem was first brought to my attention by Ben Stragnell (then at Naughty Dog). It was also pointed out by Alberto Ganesh Barbati.

Errata 3/1/07

pages 197-198. The routine **IntersectSegmentCylinder()** has a number of problems. The first problem is that the line that reads:

```
return k + 2 * t * (mn + t * nn) <= 0.0f;
```

should read:

```
return k + t * (2.0f * mn + t * nn) <= 0.0f;
```

This line corresponds to the expression:

$\text{Dot}(S(t) - p, S(t) - p) \leq r^2$

With $S(t) = sa + t(sb - sa)$, $m = sa - p$, $n = sb - sa$, $nn = \text{Dot}(n,n)$, $mn = \text{Dot}(m,n)$, and $k = \text{Dot}(m,m) - r^2$ we can rewrite the expression as follows:

$$\begin{aligned} \text{Dot}(S(t) - p, S(t) - p) &\leq r^2 \\ \text{Dot}(m + t*n, m + t*n) &\leq r^2 \\ \text{Dot}(m,m) + 2*t*\text{Dot}(m,n) + t^2*\text{Dot}(n,n) &\leq r^2 \\ \text{Dot}(m,m) - r^2 + 2*t*\text{Dot}(m,n) + t^2*\text{Dot}(n,n) &\leq 0 \\ k + 2*t*mn + t^2*nn &\leq 0 \\ k + t * (2*mn + t*nn) &\leq 0 \end{aligned}$$

The text should probably have given this derivation, as well as the derivation for the line that reads:

```
return k + dd - 2 * md + t * (2 * (mn - nd) + t * nn) <= 0.0f;
```

This line corresponds to the expression:

$\text{Dot}(S(t) - q, S(t) - q) \leq r^2$

With $S(t) = sa + t(sb - sa)$, $m = sa - p$, $n = sb - sa$, $d = q - p$, $nn = \text{Dot}(n,n)$, $mn = \text{Dot}(m,n)$, $dd = \text{Dot}(d,d)$, $md = \text{Dot}(m,d)$, $nd = \text{Dot}(n,d)$, and $k = \text{Dot}(m,m) - r^2$ we can rewrite the expression as follows:

$$\begin{aligned} \text{Dot}(S(t) - q, S(t) - q) &\leq r^2 \\ \text{Dot}(sa - q + t*n, sa - q + t*n) &\leq r^2 \\ \text{Dot}(m - d + t*n, m - d + t*n) &\leq r^2 \\ \text{Dot}(m, m - d + t*n) - \text{Dot}(d, m - d + t*n) + t*\text{Dot}(n, m - d + t*n) &\leq r^2 \\ \text{Dot}(m,m) - \text{Dot}(m,d) + t*\text{Dot}(m,n) - \text{Dot}(d,m) + \text{Dot}(d,d) - t*\text{Dot}(d,n) + t*\text{Dot}(n,m) - t*\text{Dot}(n,d) + t^2*\text{Dot}(n,n) &\leq r^2 \\ \text{Dot}(m,m) - 2*\text{Dot}(m,d) + 2*t*\text{Dot}(m,n) + \text{Dot}(d,d) - 2*t*\text{Dot}(d,n) + t^2*\text{Dot}(n,n) &\leq r^2 \\ \text{Dot}(m,m) - r^2 + \text{Dot}(d,d) - 2*\text{Dot}(m,d) + 2*t*\text{Dot}(m,n) - 2*t*\text{Dot}(d,n) + t^2*\text{Dot}(n,n) &\leq 0 \\ k + dd - 2*md + 2*t*mn - 2*t*nd + t^2*nn &\leq 0 \\ k + dd - 2*md + t*(2*mn - 2*nd + t*nn) &\leq 0 \\ k + dd - 2*md + t*(2*(mn - nd) + t*nn) &\leq 0 \end{aligned}$$

In code this would correspond to what I have in the book, except for the sake of consistency the floating point numbers should have an 'f' suffix:

```
return k + dd - 2.0f * md + t * (2.0f * (mn - nd) + t * nn) <= 0.0f;
```

Worse is that the control flow on page 198 is messed up, starting from (and including) the if-statement testing if `t < 0.0f || t > 1.0f`. **I have *not* had a chance to verify it**, but off the top of my head I believe the control flow on page 198 should read something like:

```
float t0 = t = (-b Sqrt(discr)) / a;
if (md + t * nd < 0.0f) {
    // Intersection outside cylinder on 'p' side
    if (nd <= 0.0f) return 0; // Segment pointing away from endcap
    t = -md / nd;
    // Keep intersection if Dot(S(t) - p, S(t) - p) <= r^2
    if (k + t * (2.0f * mn + t * nn) <= 0.0f) return 1;
} else if (md + t * nd > dd) {
    // Intersection outside cylinder on 'q' side
    if (nd >= 0.0f) return 0; // Segment pointing away from endcap
    t = (dd - md) / nd;
    // Keep intersection if Dot(S(t) - q, S(t) - q) <= r^2
    if (k + dd - 2.0f * md + t * (2.0f * (mn - nd) + t * nn) <= 0.0f) return 1;
}
t = t0;
// Intersection if segment intersects cylinder between the end-caps
return t >= 0.0f && t <= 1.0f;
```

Until I've had a chance to definitively correct the code, I'm interested in reports confirming (or denying) that this suggested code change addresses the problems.

Thanks to both Aaron Ondek (Vicarious Visions) and Charles Warren for pointing out these issues.

Errata 11/10/06

pages 224-226. This is not exactly an error, but a clarification. The two different versions of **TestMovingSphereSphere** given on pages 224 and 225 are not exactly identical. The first routine (page 224) does not actually test that the *t*-value lies in the [0..1] range, whereas the second routine (pages 225-226) does.

The result of "1" in the former routine just means that the spheres *will* intersect at some future time, as given by *t*. In the latter routine a result of "1" is only given if the *t* value lies within the bounds established by the movement vectors **v**₀ and **v**₁.

Simon Harrison (Si Design) pointed out this discrepancy.

Errata 8/19/06

page 211. The expression in the sentence "The last case (5) can then be identified by **n**₁ · (**n**₂ × **n**₃) = 0" is incorrect. The correct expression should feature a not-equal sign and thus read: **n**₁ · (**n**₂ × **n**₃) ≠ 0.

Thanks to Adrian Stephens of Isopod Games for reporting this error.

Errata 7/6/06

page 294. The first code line of the while-loop in the function **TestObjectAgainstGrid()** reads:

```
int32 objectMask = objectsMask & (objectsMask - 1);
```

This incorrectly strips off the low order bit from the number rather than returning all bits but the low order bit cleared. The line should read:

```
int32 objectMask = objectsMask & -objectsMask;
```

This error was communicated by Danilo Horta of Brazil.

NOTE: All errors below were corrected in the current, second printing of RTCD.

Errata 9/12/05

page 172. In the function **TestTriangleAABB()**, the sixth line on page 172 should read:

```
p2 = v2.z*(v1.y - v0.y) - v2.y*(v1.z - v0.z);
```

The corresponding formula on the preceding page is correctly given.

page 187. The first line of both the third and fourth code block read different. In the cross product call, the one line has a second argument of **p** whereas the other line has a second argument of **q**. Technically this is not incorrect because it holds that Cross(pq, p) = Cross(q - p, p) = Cross(q, p) - Cross(p, p) = Cross(q, p) = Cross(-p, q) = Cross(pq, q). However, it is very misleading to have different expressions used in the two code snippets, so for the sake of consistency both lines should read (for example):

```
Vector m = Cross(pq, p);
```

Both of these errors were reported by Tatsuya Nakamura (who is also the translator of the Japanese edition of RTCD).

Errata 7/29/05

pages 103-104. In the function **TestOBBOBB()**, the fourth line on page 104 incorrectly makes a reference to `a.u[2]` where the vector `a.u[1]` should be referenced. That is, the line should read:

```
t = Vector(Dot(t, a.u[0]), Dot(t, a.u[1]), Dot(t, a.u[2]));
```

Robert Fleming reported this error.

Errata 7/28/05

page 17. In Section 2.4.3, on page 17, the fourth sentence of paragraph two should read: "Thus, intersection of the swept volumes is a necessary but not sufficient, condition for

object collision." That is, I managed to swap necessary and sufficient around. (I hate when that happens.)

Thanks to Joe Collman for reporting the error.

Errata 6/7/05

page 476. An asterisk (*) is missing in front of the variable f2 in struct WingedEdge. The corrected struct should read:

```
struct WingedEdge {
    Vertex *v1, *v2;           // The two vertices of the represented edge (E)
    Face *f1, *f2;             // The two faces connected to the edge
    Edge *e11, *e12, *e21, *e22; // The next edges CW and CCW for each face
};
```

Errata 4/27/05

page 181. In the function **IntersectRayAABB()** the line comparing t2 against tmax has the comparison backward: the greater-than sign should be a less-than sign. In other words, the corrected line should read:

```
if (t2 < tmax) tmax = t2;
```

Thanks to Fabio Nakamura of Brazil for pointing this out.

Note that a probably better way of writing the two comparisons of t1 and t2 against tmin and tmax is as:

```
tmin = Max(tmin, t1); // Rather than: if (t1 > tmin) tmin = t1;
tmax = Min(tmax, t2); // Rather than: if (t2 < tmax) tmax = t2;
```

The **Min()** and **Max()** expressions are more explanatory and have the added benefit that in the cases where they compile to native floating-point instructions two (often expensive) branches are avoided.

Errata 4/13/05

pages 326-327. In the function **VisitCellsOverlapped()** the two lines calculating tx and ty are incorrect. The less-than sign in each line should be a greater-than sign. That is, the two lines should read:

```
float tx = ((x1 > x2) ? (x1 - minx) : (maxx - x1)) / Abs(x2 - x1);
float ty = ((y1 > y2) ? (y1 - miny) : (maxy - y1)) / Abs(y2 - y1);
```

Thanks to Jetro Lauha of Fathammer in Helsinki, Finland for reporting this error.

Jetro also points out that the computations of i, j, iend, and jend are incorrectly rounded if the line coordinates are allowed to go negative. While that was not really the intent of the code — that is, I assumed grids to be numbered from (0, 0) to (m, n) — I'm at fault for not making my assumption clear. Where it is important to handle negative line coordinates the computation of these variables should be changed to something like this:

```
// Determine start grid cell coordinates (i, j)
int i = (int)floorf(x1 / CELL_SIDE);
int j = (int)floorf(y1 / CELL_SIDE);

// Determine end grid cell coordinates (iend, jend)
int iend = (int)floorf(x2 / CELL_SIDE);
int jend = (int)floorf(y2 / CELL_SIDE);
```

Errata 2/23/05

page 312. The righthand side of the test expression of the first if-statement of the **InsertObject()** function (whew) contains a superfluous term, resulting in an incorrect test. (Additionally, for robustness, the comparison should probably be less-equal rather than less-than.) The corrected if-statement should therefore read:

```
...
if (Abs(delta) <= pObject->radius) {
    ...
```

This error was communicated by the very excellent Swaminathan Narayanan of Naughty Dog.

Errata 2/6/05

page 230. The three question mark/comma expressions of the function **Corner()** all incorrectly have their test expression reading **(n & 1)**. The corrected function should read:

```
Point Corner(AABB b, int n)
{
    Point p;
    p.x = (n & 1) ? b.max.x : b.min.x;
    p.y = (n & 2) ? b.max.y : b.min.y;
    p.z = (n & 4) ? b.max.z : b.min.z;
    return p;
}
```

Thanks to Mark Wayland of Torus Games in Melbourne, Australia for reporting this error.

Errata 1/28/05

page 187. The expressions " $e_0 = c - b$ " and " $e_1 = a - c$ " are set incorrectly. They should read " $\mathbf{e}_0 = C - B$ " and " $\mathbf{e}_1 = A - C$ " respectively. (That is, the two e 's should be set in boldface with no italics, and the four occurances of " a ," " b ," and " c " should be set in uppercase.)

Errata 1/9/05

page 153. The expressions set in boldface should all additionally be set in the code font (eight expressions in all).

Errata 1/7/05

All illustrations in the book were redrawn from my original illustrations. Unfortunately, the company in charge of redrawing the illustrations mangled almost all of the illustrations doing so (right angles became accute or obtuse, etc). I thought I had caught all their errors before the book went to print but sadly it seems I missed a few. These in particular:

page 271. The rightmost vertical edge of the array is missing in Figure 6.6. That is, the box enclosing the "K" should have a right side.

page 355. The lines connecting node 1 to nodes 2 and 4 (indicating that nodes 2 and 4 are subtrees of node 1) are missing from the tree on the right-hand side of Figure 8.5.

page 393. The inequalities given in Figure 9.7 were printed without less-than-or-equal signs. They should read: $-x+y \leq -1$, $-x-4y \leq -1$, $4x+y \leq 19$, $-4x+y \leq 0$, $x-4y \leq 0$, and $x+y \leq 5$. (These are the same inequalities given in the text, on pages 393-394.)

page 402. A minor error, but in the fourth (right bottommost) illustration of Figure 9.11 the origin is in the wrong place. It should be at the arrowhead of the nearby arrow.

page 447. In both illustrations of Figure 11.9 the line L is dashed incorrectly. On the left, L should be solid from the top all the way to the point P1. On the right, the dashing of L should start immediately at P2 (going down).